

SALESFORCE

Multi-Line Insurance Policy & Claims Management System

Implementation Report

Platform

Salesforce Lightning / Developer Edition

Document focus

Data model, policy quoting, claims routing, approval workflow, Apex/LWC components, security, and implementation evidence

1. EXECUTIVE SUMMARY

The Multi-Line Insurance Policy and Claims Management System is a Salesforce-based implementation designed to centralize insurance policy data, guide agents through policy-specific quoting, automate premium calculation, route claims, and provide a focused claims workspace. The implementation covers three policy lines: Auto, Property, and Life. A shared Policy object is adapted through record types and field sets, while the Claim object supports policy-specific processing and approval states.

The solution combines declarative configuration with programmatic components. Screen Flows handle guided quoting; Record-Triggered Flows handle claim routing and submission actions; Apex supplies premium calculation and the claim data service; Lightning Web Components present assigned claims in a filterable interface. The result is an integrated workflow from policy capture through claim review and approval.

1.1 Scope at a Glance

Area	Implementation	Purpose
Policy management	Policy custom object; Auto, Property, Life record types; field sets	Represent multiple insurance products in one structured model
Claim management	Claim object; Accident, Property, Life record types	Capture claim details and classify processing paths
Quoting	AutoQuotingFlow	Guide entry of customer, start date, state, VIN, and model year
Rating	PremiumCalculator Apex	Calculate and return policy premium from stored policy attributes
Routing	Record-Triggered Flow	Assign claims to Auto, Property, or Life queues
Approval	High Value Claim Approval	Route claims above the documented amount threshold through approval
Adjuster workspace	ClaimsAdjusterController + LWC	Show assigned claims with policy/customer context and filters
Data quality	VIN validation rule and structured picklists	Prevent invalid or incomplete policy data
Access control	Profiles, permission sets, object permissions	Provide role-specific access to policy and claim data

2. BUSINESS REQUIREMENTS AND DESIGN GOALS

The project guide identifies three primary business needs: standardize and accelerate quoting and policy issuance, automate claim initiation and routing using policy type and claim severity, and maintain a 360-degree view of customer, policy, and claim information. These requirements drive the architecture rather than treating each insurance product as a separate application.

2.1 Functional Requirements

1. Support Vehicle/Auto, Property, and Life insurance within a shared Policy structure.
2. Capture product-specific fields through record types and field sets without duplicating the core Policy object.
3. Use a multi-screen quoting experience to collect policy holder and product information in a controlled sequence.
4. Calculate a premium after a draft policy is created and write the result back to the Policy record.
5. Route new claims to the queue associated with the Policy record type.

6. Submit high-value claims for formal approval and maintain explicit approval states.
7. Provide adjusters with a focused view of their assigned claims and basic filtering by policy type.
8. Apply validation and permission controls so only appropriate users can create or update sensitive records.

2.2 Non-Functional Goals

Goal	Design response
Consistency	Shared Policy and Claim objects with controlled record types and picklist values
Maintainability	Declarative Flow for orchestration; Apex isolated to calculations and data retrieval
Usability	Screen Flow for guided capture and LWC for claim review
Traceability	Auto-numbered Policy and Claim records, linked Customer and Policy references
Security	Permission sets and object-level access for claims-focused users
Extensibility	Policy-specific field sets allow new fields to be added without redesigning the core object

3. SOLUTION ARCHITECTURE

The implementation is intentionally layered. The data layer provides common insurance records; the automation layer coordinates quoting, rating, routing, and approval; the presentation layer provides an adjuster-facing claim view. This separation keeps the system understandable while still allowing Apex to handle logic that is awkward or inefficient to express entirely through declarative configuration.

3.1 Component Map

Layer	Components	Role
Data model	Policy__c, Claim__c, Contact	Stores policy, claim, and customer context
Configuration	Record Types, Field Sets, Picklists	Differentiates Auto, Property, Life, and claim categories
Declarative automation	AutoQuotingFlow; claim-routing flow; approval submission action	Guides user input and moves records through business stages
Programmatic logic	PremiumCalculator; ClaimsAdjusterController	Performs premium calculation and claim retrieval
User interface	claimsDashboardLwc; claimTileLwc	Displays assigned claims and policy-type filtering
Security	Permission sets and object permissions	Controls what claims-focused users can see or modify

3.2 End-to-End Processing

Stage	Primary component	Input	Output
Policy capture	AutoQuotingFlow	Customer, start date, state, VIN, model year	Draft Auto Policy
Premium rating	PremiumCalculator	Policy Id and policy fields	Calculated premium
Policy update	AutoQuotingFlow	Calculated premium	Policy record updated with Premium

Stage	Primary component	Input	Output
Claim creation	Claim object	Claim amount, date of loss, policy, description	New claim
Claim routing	Record-Triggered Flow	Related Policy record type	Auto / Property / Life queue assignment
High-value approval	High Value Claim Approval	Claim Amount > 50,000	Sequential approval path
Adjuster review	ClaimsAdjusterController + LWC	Current-user claims	Filterable claim tiles with policy/customer data

4. DATA MODEL AND CONFIGURATION

The data model uses a common Policy object to represent multiple insurance lines and a Claim object to represent claim events. Policy Record Types separate Auto, Property, and Life behavior. Field Sets keep line-specific data discoverable without fragmenting the core record structure.

4.1 Policy Configuration

Element	Configuration
Object	Policy
Record name	Policy Number — Auto Number, format P - {0000}, starting number 1
Record types	Auto, Property, Life
Vehicle field set	VIN and Model Year
Property field set	Square Footage and Year Built
Life field set	Beneficiary Name and Policy Term Months
Core fields	Customer, Policy Start Date, Policy Term Months, Premium, VIN, Model Year, Square Footage, Year Built, Beneficiary Name

4.2 Claim Configuration

Element	Configuration
Object	Claim
Record name	Claim Number — Auto Number, format C - {0000}, starting number 1
Record types	Accident, Property, Life
Key fields	Adjuster, Approval Status, Claim Amount, Date of Loss, Description, Policy
Approval Status	New; Submitted for Approval; Approved; Rejected

4.3 Relationship View

Parent / reference	Relationship	Use
Claim → Policy	Lookup	Connect claim to the insured policy
Policy → Customer	Lookup to Contact	Connect policy to the policy holder
Claim → Adjuster	Lookup to User	Identify the assigned claims user
Policy → Record Type	Record type	Drive product-specific behavior and routing
Claim → Record Type	Record type	Classify claim processing context

5. POLICY QUOTING AND PREMIUM CALCULATION

5.1 AutoQuotingFlow

The AutoQuotingFlow is a Screen Flow that separates common policy information from vehicle-specific information. The first screen captures the policy holder, start date, and policy state. The second screen captures the VIN and model year. After the inputs are collected, a Create Records element creates a draft Policy and stores the resulting record Id in a flow variable.

Flow step	Configuration	Result
Get Vehicle RT ID	Get Records against Policy Record Type Auto	Provides RecordTypeId for the draft record
Basic information screen	Policy holder, start date, policy state	Captures shared inputs
Vehicle details screen	VIN, model year	Captures Auto-specific inputs
Create Draft Policy	Assigns customer, state, date, record type, VIN, model year	Creates Policy record
Calculate Premium Action	Invocable Apex action; input = policy Id	Returns calculated premium
Update Policy With Premium	Updates Premium using flow variable	Completes the draft policy record

5.2 PremiumCalculator

PremiumCalculator exposes an invocable method named Calculate Policy Premium. The implementation retrieves the policy fields required for rating, establishes a base premium, applies state adjustments for California and Texas, and adds an age-related increment for vehicle model years before 2018. The calculated amount is returned to the Flow and written to the Policy Premium field.

Rating input	Documented rule
Base premium	1,000.00
Policy State = CA	Base premium $\times$ 1.15
Policy State = TX	Base premium $\times$ 1.05
Model Year < 2018	Add 100.00

This separation is deliberate: Flow handles record orchestration and user interaction, while Apex handles the arithmetic and SOQL required by the rating step.

6. CLAIMS INTAKE, ROUTING AND APPROVAL

6.1 Claim Routing by Policy Type

A Record-Triggered Flow runs when a Claim is created. The Flow retrieves the related Policy and reads its Record Type. A Decision element then selects the routing path. Auto claims are assigned to the Auto queue, Property claims to the Property queue, and the default path assigns Life claims to the Life queue.

Decision outcome	Condition	Assignment
Auto Claim Route	Policy Record Type = Auto	Auto queue
Property Claim Route	Policy Record Type = Property	Property queue
Life Claim Route	Default outcome	Life queue

6.2 High Value Claim Approval

The project defines a formal approval process named High Value Claim Approval. Its entry condition is a Claim Amount greater than 50,000. The approval path is sequential: the claim is routed first to a Senior Adjuster and then to a Department Manager. During approval, record editing is restricted according to the documented approval configuration.

Approval element	Documented configuration
Entry criteria	Claim Amount > 50,000
Approval sequence	Senior Adjuster → Department Manager
Submitter	Claim owner is an allowed initial submitter
Approval status	New → Submitted for Approval → Approved / Rejected
Final approval action	Update Approval Status to Approved
Final rejection action	Update Approval Status to Rejected

6.3 Submission Automation

The implementation guide also defines a record-triggered submission action named Submit Claim to Approval. It passes the triggering Claim record Id to the High Value Claim Approval process, allowing the record-triggered automation to submit qualifying claims without requiring a separate manual navigation step.

7. APPROVAL WORKFLOW EVIDENCE

The following screenshots document the approval area of the Salesforce org used for the project. They are included as implementation evidence rather than decorative illustrations.

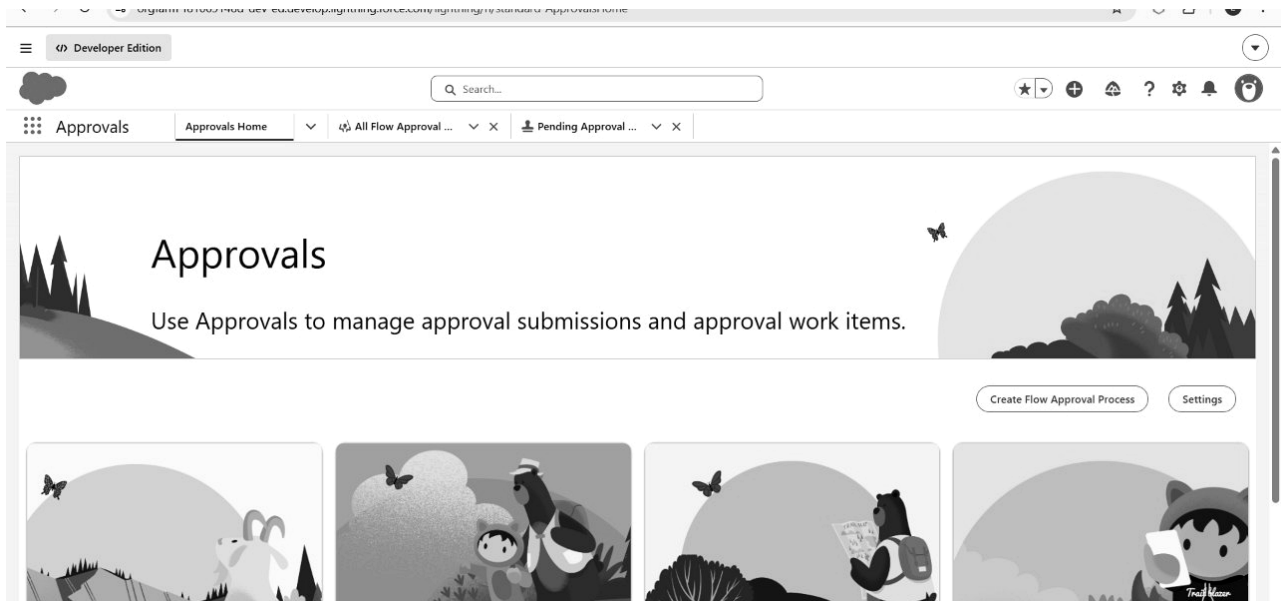


Figure 1. Salesforce Approvals Home used to access approval submissions and approval work items.

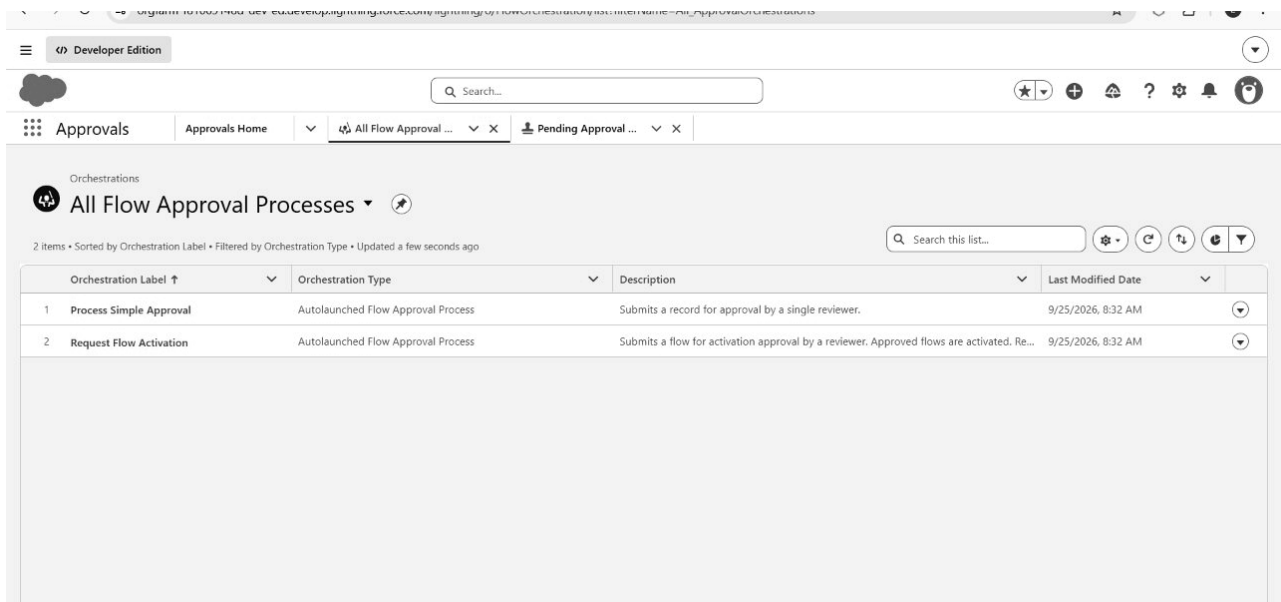


Figure 2. All Flow Approval Processes view showing the configured flow-approval processes available in the org.

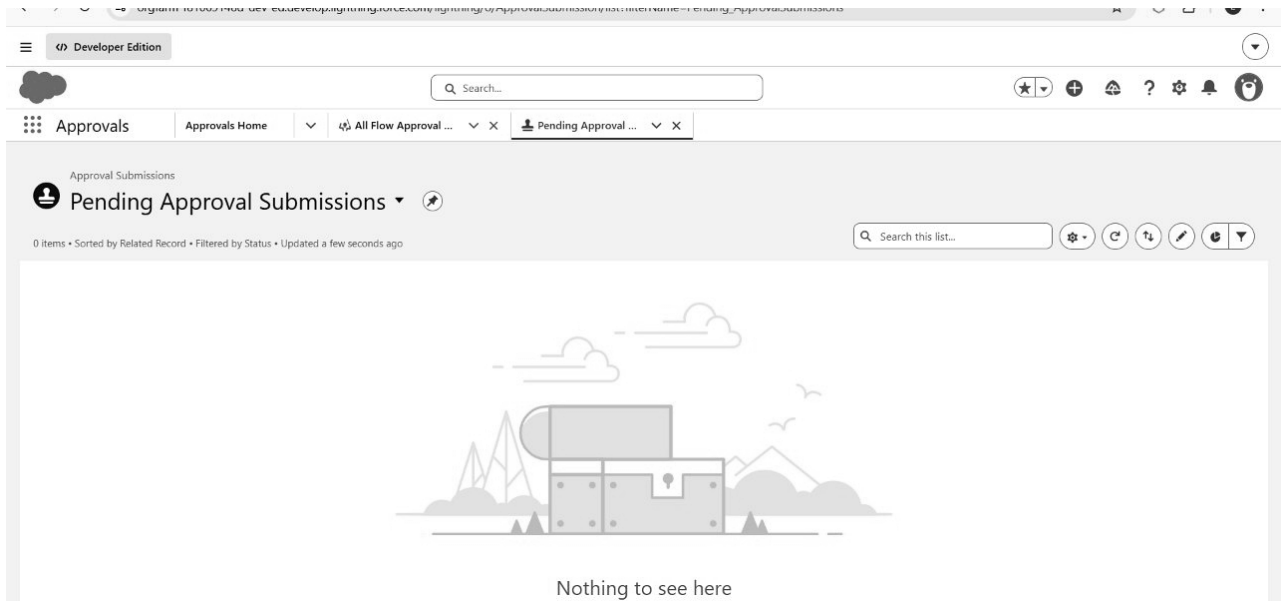


Figure 3. *Pending Approval Submissions view at capture time; no pending submission rows were present.*

7.1 Reading the Approval Screens

Screen	What it establishes
Approvals Home	The Salesforce approval workspace is available to manage submissions and work items.
All Flow Approval Processes	The org exposes flow approval processes through the approval interface.
Pending Approval Submissions	The captured list contained no pending submission rows at that moment.

8. CLAIM RECORD AND APPROVAL EVIDENCE

The next screenshots show the claim object and the approval context of a real record captured in the implementation org. Claim C-0003 is shown with a Claim Amount of 60,000 and Approval Status set to Submitted for Approval. The linked policy is P-0001.

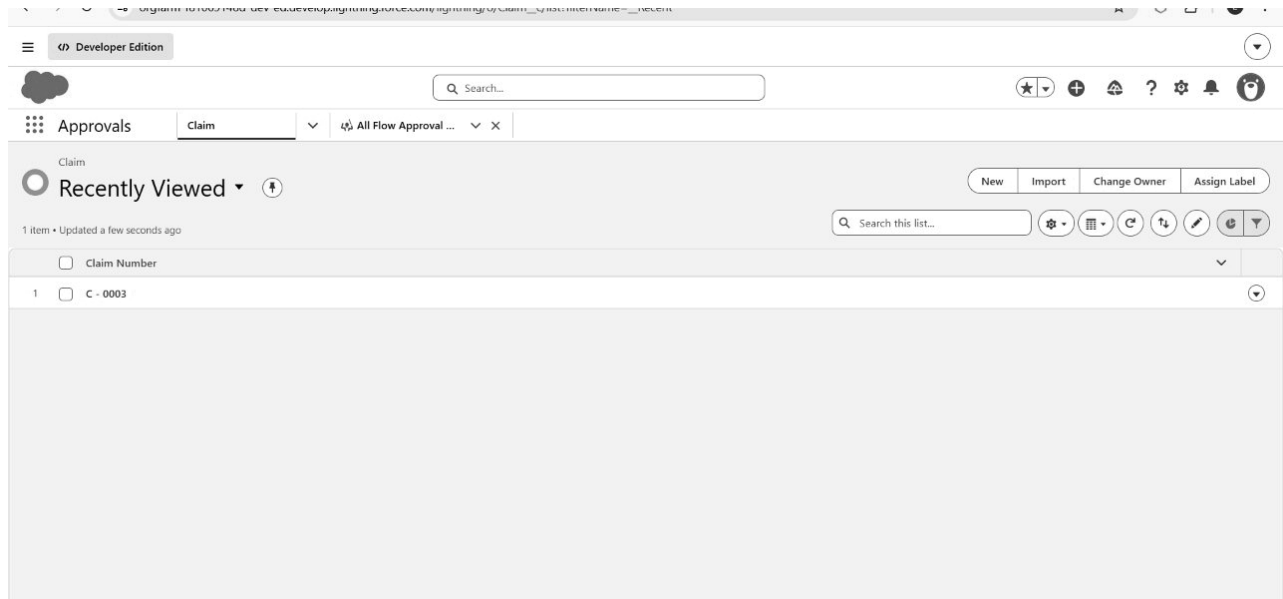


Figure 4. Claim object list view showing the captured claim record C-0003.

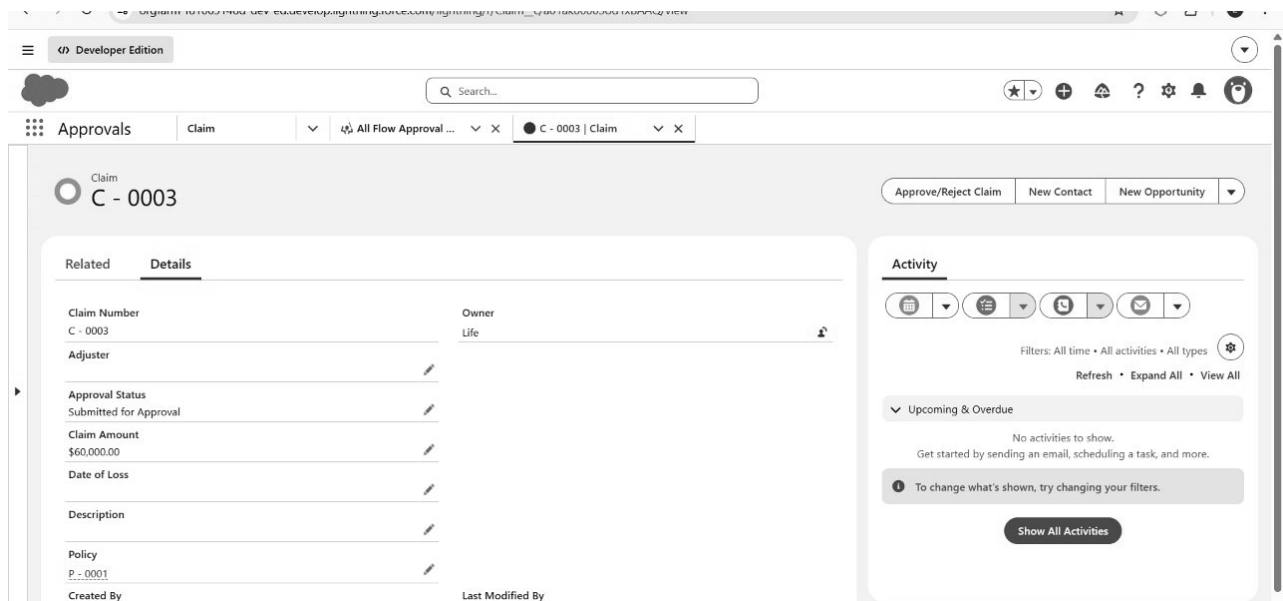


Figure 5. Claim C-0003 detail page showing the submitted-for-approval status, claim amount, and linked policy.

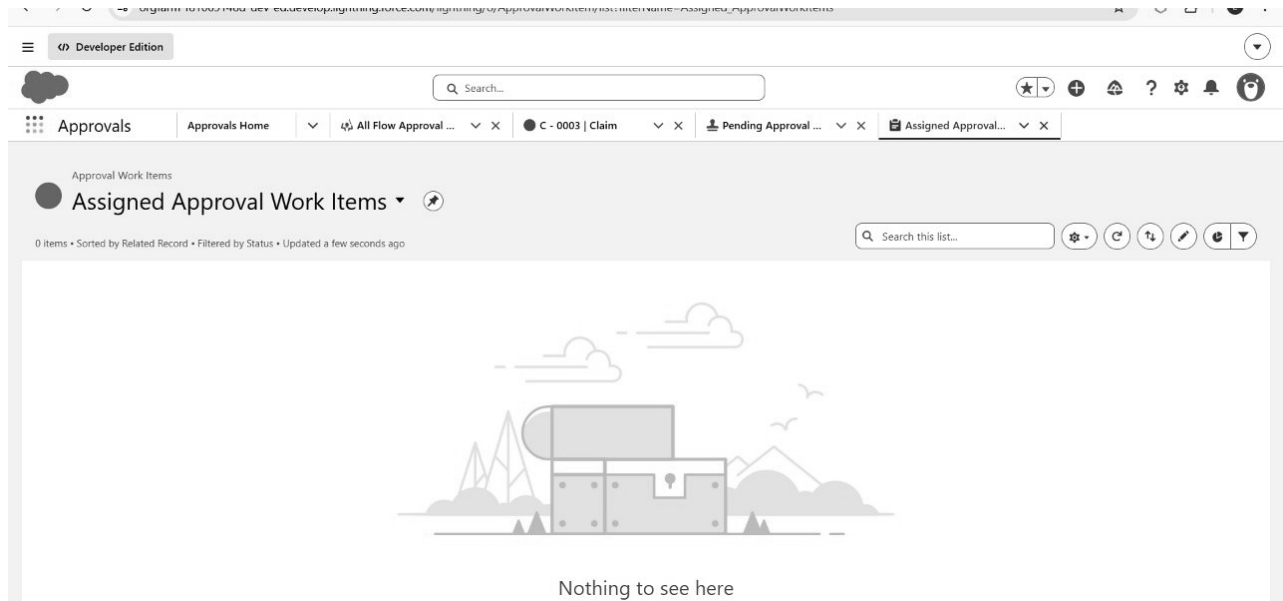


Figure 6. *Assigned Approval Work Items view at capture time; no assigned work items were listed.*

8.1 Claim Evidence Summary

Evidence	Observed state
Claim record	C-0003 is present in the Claim list.
Approval state	C-0003 shows Submitted for Approval.
Claim amount	The record shows 60,000.00.
Linked policy	The record references P-0001.
Assigned approval work items	No rows were visible at capture time.

9. CLAIMS ADJUSTER WORKSPACE

9.1 ClaimsAdjusterController

The Apex class `ClaimsAdjusterController` exposes a cacheable `getAssignedClaims` method. It queries Claims owned by the logged-in user and joins the related Policy and Customer records. Results are transformed into a `ClaimWrapper` containing the claim Id, claim number, claim amount, status, policy type, policy holder name, and days open.

Wrapper field	Purpose
claimId	Unique claim identifier
claimNumber	Human-readable claim number
claimAmount	Amount under review
status	Current approval state
policyType	Auto / Property / Life context from Policy record type
policyHolderName	Customer context from the related Contact
daysOpen	Age of the claim calculated from Date of Loss

9.2 Lightning Web Components

The `claimsDashboardLwc` component uses the Lightning wire service to load assigned claims from Apex and applies client-side filtering. The filter supports All, Auto, Property, and Life. Each result is passed to the reusable `claimTileLwc` component, which displays the claim number, policy type, policy holder, claim amount, status, and days open.

Component	Responsibility
<code>claimsDashboardLwc</code>	Retrieve assigned claims, maintain <code>visibleClaims</code> , and filter by policy type
<code>claimTileLwc</code>	Render one claim as a reusable summary tile with policy-aware iconography
<code>ClaimsAdjusterController</code>	Supply claim data through Apex and a wrapper model

10. DATA QUALITY AND ACCESS CONTROL

10.1 Validation

The project includes a VIN validation rule for Auto policies. The documented formula checks the Policy Record Type and requires the VIN length to equal 17 characters. The error is placed directly on the VIN field so the correction is visible at the point of entry.

Rule	Condition	Outcome
VIN_Must_Be_17_Characters	Auto Policy AND LEN(VIN) ≠ 17	Block save and display a field-level error on VIN

10.2 Permission Sets

The guide defines Claims Manager Access and Claims Adjuster Access permission sets. The documented settings provide more granular access without changing the base profile. Claims-focused access includes read-only Policy visibility and elevated Claim access, together with the appropriate Apex class and system permissions for the intended role.

Access set	Documented permissions
Claims Manager Access	Create/Read Policy; Read/Edit Contact; Read-only Claim; Flow User; PremiumCalculator Apex class
Claims Adjuster Access	Read-only Policy; Read/Edit/Delete Claim; Run Reports; Manage Approvals; ClaimsAdjusterController Apex class

10.3 Security Principle

The security model follows least-privilege intent: users receive only the object and Apex access required for their responsibilities, while the approval configuration controls who may submit and review high-value claims. This separates ordinary record access from approval authority.

11. IMPLEMENTATION MATRIX

The project can be reviewed as a compact mapping between business capability, Salesforce configuration, and evidence. This format is intended to make evaluation straightforward without reproducing the setup guide line by line.

Capability	Salesforce implementation	Evidence / output
Multi-line policy storage	Policy__c + record types + field sets	Auto / Property / Life policy structures
Auto quotation	AutoQuotingFlow	Draft policy creation from guided screens
Premium calculation	PremiumCalculator + Flow action	Premium returned and written to Policy
Claim classification	Claim__c + record types	Accident / Property / Life claim records
Claim routing	Record-Triggered Flow + queue assignment	Auto / Property / Life routing paths
High-value approval	High Value Claim Approval	Amount threshold and two-step review
Claim review UI	ClaimsAdjusterController + LWC	Assigned claims and policy-type filter
Data validation	VIN_Must_Be_17_Characters	Field-level enforcement on Auto policy VIN
Role-based access	Permission sets and object permissions	Claims Manager / Claims Adjuster access

12. VERIFICATION AND TEST CASES

The implementation guide is configuration-oriented, so verification is expressed here as observable business rules and expected results. The six supplied screenshots provide direct evidence of the Salesforce approval area and the captured claim record.

Test case	Input / condition	Expected result	Evidence status
T01 — VIN validation	Auto policy with VIN length other than 17	Save is blocked at VIN field	Implemented
T02 — Premium calculation	Valid Auto policy data	Apex action returns premium; Policy Premium is updated	Implemented
T03 — Claim routing	Claim linked to Auto / Property / Life policy	Claim Owner moves to corresponding queue path	Implemented
T04 — High-value threshold	Claim Amount > 50,000	Claim qualifies for High Value Claim Approval	Implemented

Test case	Input / condition	Expected result	Evidence status
T05 — Approval status	Submission and final decision	Status moves through Submitted for Approval and Approved / Rejected	Implemented
T06 — Claims dashboard	Current user has assigned claims	Claims are loaded and filterable by policy type	Implemented
T07 — Screenshot evidence	C-0003 captured at 60,000	Claim record visibly shows Submitted for Approval	Observed in supplied screenshot
T08 — Approval work items	Assigned Approval Work Items view	Work item list reflects current assignment state	Observed in supplied screenshot

13. IMPLEMENTATION NOTES AND BOUNDARIES

- The report describes the implementation documented in the supplied project guide and the supplied Salesforce screenshots. It does not infer undocumented production deployment, performance measurements, or business outcomes.
- The High Value Claim Approval threshold is documented as greater than 50,000; the supplied claim screenshot shows a 60,000 claim in Submitted for Approval status.
- The approval evidence screenshots are point-in-time captures. The Pending Approval Submissions and Assigned Approval Work Items views both show no rows at capture time; this report does not infer the reason for that state.
- The screenshot set focuses on approvals and claim handling. It is therefore used to evidence the approval path and claim record stage rather than every component in the overall system.

14. CONCLUSION

The Multi-Line Insurance Policy and Claims Management System demonstrates a coherent Salesforce implementation for handling multiple insurance product lines within a shared CRM model. Policy records are differentiated through record types and field sets, quoting is guided through Flow, premium calculation is isolated in Apex, claim routing is automated by policy type, and high-value claims are governed through sequential approval.

The claims workspace extends the platform beyond data capture by combining Apex retrieval with Lightning Web Components for an adjuster-oriented interface. The supplied Salesforce screenshots further demonstrate that the approval application, claim record, and approval work-item areas are present and accessible in the working org.

Taken together, the implementation provides a clear separation between configuration, automation, programmatic logic, and user interface. That structure keeps the project explainable for evaluation while leaving a practical foundation for future extension of policy products, claim workflows, reporting, and integration.

Appendix A. Core Component Inventory

Component	Type	Purpose
Policy__c	Custom object	Shared policy data model
Claim__c	Custom object	Claim lifecycle data
Auto / Property / Life	Policy record types	Product-specific policy behavior
Accident / Property / Life	Claim record types	Claim classification
Vehicle / Property / Life	Policy field sets	Product-specific field presentation
AutoQuotingFlow	Screen Flow	Guided Auto quote creation

Component	Type	Purpose
PremiumCalculator	Apex class	Premium calculation
Claim routing flow	Record-Triggered Flow	Policy-type queue routing
High Value Claim Approval	Approval Process	Sequential approval for claims > 50,000
ClaimsAdjusterController	Apex class	Current-user claim retrieval
claimsDashboardLwc	LWC	Claim dashboard and filtering
claimTileLwc	LWC	Reusable claim summary component
VIN_Must_Be_17_Characters	Validation Rule	Auto policy VIN validation
Claims Manager Access	Permission Set	Manager-level access
Claims Adjuster Access	Permission Set	Adjuster-level access

APPENDIX B. POLICY FIELD CATALOGUE

The Policy object is the common insurance record for the three supported product lines. The source guide specifies a compact set of common and product-oriented fields. The table below consolidates those definitions into a report-oriented data dictionary.

Field	Data type	Role in the solution	Primary use
Policy Number	Auto Number: P-{0000}	System identifier	Unique policy reference
Beneficiary Name	Text(50)	Life-specific attribute	Stores beneficiary information for Life policies
Customer	Lookup(Contact)	Customer relationship	Links the policy to the policy holder
Model Year	Text(5)	Auto-specific attribute	Stores vehicle model year
Policy Start Date	Date	Common policy field	Records policy start date
Policy Term Months	Number(18,0)	Life-specific attribute	Stores term duration
Premium	Currency(16,2)	Calculated business value	Stores the premium returned by rating logic
Square Footage	Number(18,0)	Property-specific attribute	Stores property size
VIN	Text(20)	Auto-specific attribute	Stores Vehicle Identification Number
Year Built	Text(5)	Property-specific attribute	Stores property construction year

The common-object approach keeps customer linkage and policy lifecycle information centralized while the line-specific fields are exposed through the relevant record type and field set. This avoids creating three separate policy objects for the same business concept.

APPENDIX C. POLICY LINE DESIGN

The implementation differentiates insurance products primarily through Record Types and Field Sets. This is a configuration choice that preserves a single Policy object while allowing the user interface and validation logic to recognize product context.

Policy line	Record type	Field set	Product-specific data
Auto	Auto	Vehicle	VIN; Model Year
Property	Property	Property	Square Footage; Year Built
Life	Life	Life	Beneficiary Name; Policy Term Months

Design implications

- Common policy fields remain available across product lines, while the field set determines which additional attributes are presented for the selected product.
- The Policy record type is also reused by claim-routing logic, making the insurance line available as an automation input.
- The same object can therefore support reporting and customer relationships without duplicating the core data structure.

The source guide specifically documents Auto, Property, and Life as the Policy record types and documents corresponding field sets for vehicle, property, and life data. This report preserves that terminology rather than introducing alternate product names.

APPENDIX D. CLAIM FIELD CATALOGUE

The Claim object is the operational record for loss or incident processing. In addition to classification by Record Type, the object stores the amount under review, ownership, dates, description, approval state, and policy reference.

Field	Data type	Business purpose	Control / use
Claim Number	Auto Number: C-{0000}	Unique claim reference	System-generated identifier
Adjuster	Lookup(User)	Identifies assigned adjuster	Supports ownership and review
Approval Status	Picklist	Tracks approval state	New; Submitted for Approval; Approved; Rejected
Claim Amount	Currency(16,2)	Financial value of claim	Used by high-value approval criteria
Date of Loss	Date/Time	Records incident timing	Used for claim context and aging
Description	Long Text Area	Narrative incident detail	Stores descriptive information
Policy	Lookup(Policy)	Links claim to insured policy	Used by routing and dashboard logic

Claim record types

Record type	Routing implication
Accident	Documented as the base accident claim type in the guide
Property	Supports property-oriented claim processing
Life	Supports life-oriented claim processing and the default routing path

APPENDIX E. RECORD TYPES AND FIELD SETS

Record Types and Field Sets work together in this implementation. Record Types provide product context, while Field Sets define the product-specific fields that should be surfaced for that context. The same design principle is applied to claims through claim record types.

Configuration area	Record Type	Field Set / key fields	Why it matters
Policy	Auto	Vehicle: VIN, Model Year	Allows vehicle-specific capture within Policy
Policy	Property	Property: Square Footage, Year Built	Captures property-specific attributes
Policy	Life	Life: Beneficiary Name, Policy Term Months	Captures life-policy attributes
Claim	Accident	Adjuster, Approval Status, Claim Amount, Date of Loss	Supports accident claim lifecycle
Claim	Property	Same core claim fields with Property record context	Supports property-specific routing
Claim	Life	Same core claim fields with Life record context	Supports life-specific routing

Implementation principle

This configuration avoids hard-coding product-specific fields into separate objects and keeps routing logic based on a stable Salesforce construct: the Record Type associated with the Policy. The result is a compact model that can be extended with additional fields or product types without rewriting the entire data layer.

APPENDIX F. AUTOQUOTINGFLOW: SCREEN DESIGN

AutoQuotingFlow is documented as a multi-screen Screen Flow. It first identifies the Auto Policy Record Type, then collects common policy information, followed by vehicle-specific data, and finally creates the draft Policy record.

Screen / step	Inputs or resources	Purpose
Get Vehicle RT ID	Policy object; Record Type Name = Auto	Find the Auto Record Type Id
Screen 1 — Policy Basics	Customer lookup; Policy Start Date; Policy State	Capture common policy information
Screen 2 — Vehicle Details	VIN; Model Year	Capture Auto-specific data
Create Draft Policy	Customer; Model Year; Start Date; Policy State; Record Type Id; VIN	Persist the quotation as a Policy record
Policy Id variable	varPolicyId text variable	Expose the created Policy Id to subsequent actions

User interaction sequence

- The flow separates common fields from vehicle-specific fields so the user is not shown a single dense screen.
- The created record is retained as a draft Policy, providing a stable record Id for premium calculation.
- The flow is then extended with an Apex action and an update step to write the calculated premium back to the same record.

APPENDIX G. AUTOQUOTINGFLOW: DATA MAPPING

The following mapping summarizes the documented field assignments used when the draft Auto Policy is created. This is the key hand-off between the screen layer and the Salesforce data layer.

Policy field	Source in Flow	Type / note
Customer	Basic information > Policy Holder > Record ID	Lookup(Contact)

Policy field	Source in Flow	Type / note
Model Year	Vehicle information screen > Model Year	Text
Policy Start Date	Basic information > Policy Start Date	Date
Policy State	Basic information > Policy State	Picklist choice set
Record Type ID	Get Vehicle RT ID > Record Type ID	Identifies Auto policy line
VIN	Vehicle information screen > VIN	Text
varPolicyId	Created Policy record Id	Text variable for later actions

The Flow therefore performs two distinct jobs: it collects user input and establishes a persistent Policy record. The subsequent Apex action does not need to understand the screen controls themselves; it receives a Policy Id and reads the stored fields from Salesforce.

APPENDIX H. PREMIUM CALCULATION SPECIFICATION

PremiumCalculator is an invocable Apex class integrated into AutoQuotingFlow. The documented method accepts a Policy Id, retrieves VIN, Model Year, and Policy State, calculates a base premium, and returns a Decimal value to Flow.

Rating input	Condition	Documented adjustment
Base premium	Always	1,000.00
Policy State	CA	Multiply base premium by 1.15
Policy State	TX	Multiply base premium by 1.05
Model Year	Earlier than 2018	Add 100.00

Processing sequence

Step	Operation
1	Collect Policy Id from the Flow variable varPolicyId
2	Query Policy__c for VIN__c, Model_Year__c, and Policy_State__c
3	Start with basePremium = 1000.00
4	Apply state-specific multiplier where applicable
5	Apply the model-year adjustment where applicable
6	Return the calculated Decimal to Flow
7	Update Policy.Premium with the returned value

APPENDIX I. CLAIM ROUTING SPECIFICATION

Claim routing is implemented as a Record-Triggered Flow configured for the Claim object. When a claim is created, the Flow retrieves the related Policy and reads the Policy Record Type. A Decision element selects the appropriate routing outcome.

Decision outcome	Condition	Assignment
Auto Claim Route	varPolicyRTName = Auto	Auto queue
Property Claim Route	varPolicyRTName = Property	Property queue
Life Claim Route	Default outcome	Life queue

Routing sequence

- Claim creation starts the record-triggered automation.
- Get_Policy_RT retrieves the Policy referenced by the Claim.
- The Record Type identifier is stored in varPolicyRTName for decision logic.
- The Decision element routes the record to an insurance-line-specific queue.
- The Claim record is updated by setting OwnerId to the selected queue Id.

The documented implementation uses queue assignment rather than embedding an individual adjuster in the initial routing rule. This creates a stable operational hand-off where claims can be processed by the appropriate team.

APPENDIX J. HIGH-VALUE CLAIM APPROVAL

The project introduces a separate approval process for high-value claims. The process is named High Value Claim Approval and is entered when the Claim Amount exceeds 50,000. The documented sequence routes the claim first to a Senior Adjuster and then to a Department Manager.

Approval element	Documented configuration
Entry criteria	Claim Amount > 50,000
Approval order	Senior Adjuster → Department Manager
Initial submitters	Claim owner is configured as an allowed submitter
Record editability	Administrators only during approval
Approval result	Approval Status updated to Approved
Rejection result	Approval Status updated to Rejected

Business control represented

The workflow separates ordinary claim intake from escalated review. A claim can be created and routed normally, but a claim above the documented threshold enters a controlled approval path before its final status is marked Approved. The approval status field therefore acts as both a user-facing state and a workflow output.

APPENDIX K. APPROVAL STATE MODEL

The Approval Status picklist is documented with four values: New, Submitted for Approval, Approved, and Rejected. These values provide a compact state model for the claim approval lifecycle.

State	Meaning in the implementation	Typical transition
New	Claim exists but has not been submitted for approval	Created → New
Submitted for Approval	Claim has entered the approval process	New → Submitted for Approval
Approved	Final approval action has completed	Submitted for Approval → Approved
Rejected	Final rejection action has completed	Submitted for Approval → Rejected

Action mapping

Action	Target field	New value
Submission	Approval Status	Submitted for Approval
Final approval	Approval Status	Approved

Action	Target field	New value
Final rejection	Approval Status	Rejected

This state model also provides a clear basis for dashboard display and test cases. The supplied claim screenshot confirms the Submitted for Approval state on claim C-0003.

APPENDIX L. AUTOMATED APPROVAL SUBMISSION

The guide also defines a record-triggered submission action named Submit Claim to Approval. The action passes the triggering Claim Record Id to the High Value Claim Approval process. This creates a link between record-triggered automation and the approval engine.

Configuration item	Documented value
Action label	Submit Claim to Approval
Record Id input	Triggering Claim → Record ID
Approval process	High Value Claim Approval
Purpose	Submit the qualifying Claim into the configured approval process

Why the action is useful

- It keeps the submission step within the same automation framework as claim processing.
- It reduces dependency on separate navigation to an approval screen for qualifying records.
- It makes the approval process callable from the claim lifecycle rather than treating approval as an isolated administration step.

APPENDIX M. CLAIMSADJUSTERCONTROLLER DATA CONTRACT

ClaimsAdjusterController provides the server-side data contract consumed by the claims dashboard. The wrapper class exposes the small set of attributes needed for a claim summary view, keeping the user interface independent from the full Salesforce object shape.

Wrapper attribute	Source / calculation	UI use
claimId	Claim__c.Id	Component key
claimNumber	Claim__c.Name	Display heading
claimAmount	Claim_Amount__c	Financial value
status	Approval_Status__c	Workflow state
policyType	Policy Record Type name	Filter and context
policyHolderName	Policy.Customer__r name	Customer context
daysOpen	Date.today() compared with Date_of_Loss	Aging indicator

Query scope

The documented SOQL query filters Claim__c by OwnerId = UserInfo.getUserId() and orders results by Date_of_Loss__c descending. The query joins the related Policy Record Type and the customer first and last names, after which the results are wrapped into ClaimWrapper instances.

APPENDIX N. CLAIMS DASHBOARD COMPONENT DESIGN

The claims dashboard is split into a parent component and a reusable tile component. The parent owns data retrieval and filtering; the tile owns presentation of one claim record. This is a straightforward component boundary for Lightning Web Components.

Component	Responsibility	Input / output
claimsDashboardLwc	Call Apex through @wire; store allClaims and visibleClaims; expose filter options	Receives server data; passes each visible claim to tile component
claimTileLwc	Render one claim; determine policy icon based on policy type	Receives claimData from parent
ClaimsAdjusterController	Query assigned claims and build wrapper objects	Returns List<ClaimWrapper>

Supported policy filter values

Label	Value
All Policy Types	All
Auto	Auto
Property	Property
Life	Life

The filter is intentionally client-side after the initial Apex retrieval. Selecting All restores the full collection; selecting a product line applies a simple equality filter on claim.policyType.

APPENDIX O. CLIENT-SIDE FILTERING LOGIC

The LWC stores two collections: allClaims and visibleClaims. allClaims represents the current server response; visibleClaims represents what the user sees after a policy-type filter is applied. This pattern preserves the original data set and avoids a second server request for each filter choice.

Interaction	Component state change
Initial load	allClaims = data; visibleClaims = data
Select All	visibleClaims = allClaims
Select Auto	visibleClaims = allClaims filtered where policyType = Auto
Select Property	visibleClaims = allClaims filtered where policyType = Property
Select Life	visibleClaims = allClaims filtered where policyType = Life
Apex error	error is populated and visible lists are cleared

Presentation elements

- A Lightning Card provides the overall dashboard container.
- A combobox exposes the policy-type filter.
- Each result is rendered through claimTileLwc rather than duplicating markup in the parent.
- The page displays the total number of currently visible claims.

APPENDIX P. SECURITY AND ACCESS MATRIX

The supplied guide documents two claims-focused permission sets. The matrix below separates object access from Apex/system permissions so the access model can be reviewed independently of the user interface.

Permission set	Policy	Contact	Claim	Apex / system
Claims Manager Access	Create, Read	Read, Edit	Read only	Flow User; PremiumCalculator enabled
Claims Adjuster Access	Read only	Read, Edit	Read, Edit, Delete	Run Reports; Manage Approvals; ClaimsAdjusterControlle r enabled

Security rationale

The permission-set pattern lets the implementation add role-specific capabilities without rewriting the underlying profile. Policy visibility can remain comparatively limited while Claim access is elevated for staff who actively process claims. Apex access is also explicit, ensuring that programmatic capabilities are granted only where documented.

APPENDIX Q. VALIDATION AND DATA QUALITY CONTROLS

Data quality is handled by a targeted validation rule rather than by relying only on user training. The Auto VIN rule checks two conditions together: the current Policy record type must be Auto, and the VIN length must not differ from 17 characters.

Rule	Formula logic	User-facing outcome
VIN_Must_Be_17_Characters	AND(RecordType.DeveloperName = "Auto", LEN(VIN__c) <> 17)	Save is blocked and an explanatory message is shown on VIN

Additional integrity controls documented in the model

- Policy and Claim numbers are generated through Auto Number record names.
- Policy-specific data is separated by Record Type and Field Set rather than duplicated objects.
- Approval Status is a controlled picklist rather than free text.
- Claim routing derives from the linked Policy Record Type.
- Access to sensitive objects and Apex classes is granted through explicit permission sets.